

SKILL WEEK – 9

TASK1: To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

```
shreyaj@LAPTOP-FT6VVM4D: ~$ mkdir -p ~/Skill9
cd ~/Skill9
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ sleep 100 &
[1] 1036
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ jobs
[1]+  Running                  sleep 100 &
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ jobs -l
[1]+  1036 Running              sleep 100 &
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ jobs -p
1036
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ bg %1
-bash: bg: job 1 already in background
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ fg %1
sleep 100
^C
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ |
```

TASK2: To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

```
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ sleep 200
^Z
[1]+  Stopped                  sleep 200
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ jobs
[1]+  Stopped                  sleep 200
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ bg %1
jobs
[1]+ sleep 200 &
[1]+  Running                  sleep 200 &
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ fg %1
sleep 200
^C
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ |
```

TASK3: To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

```

shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ nano signal.c
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ gcc signal.c -o signal
./signal
Process ID: 1178
Press Ctrl+C to send SIGINT
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
^C
Signal received: 2
Running...
^C
Signal received: 2
Running...
^C
Signal received: 2
Running...

```

```

GNU nano 8.7.1 signal.c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig) {
    printf("\nSignal received: %d\n", sig);
}

int main() {
    // Intercept SIGINT (Ctrl+C) and SIGTERM (kill command)
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);

    printf("Process ID: %d\n", getpid());
    printf("Press Ctrl+C to send SIGINT\n");

    while (1) {
        printf("Running...\n");
        sleep(2);
    }

    return 0;
}

```

```

shreyaj@LAPTOP-FT6VVM4D: $ ps aux | grep signal
root      224  0.0  0.4 123464 32704 ?        Ssl  03:47   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
shreyaj   1178  0.0  0.0 2768 1948 pts/0    S+   04:14   0:00 ./signal
shreyaj   1228  0.0  0.0 4128 2408 pts/2    S+   04:15   0:00 grep --color=auto signal

shreyaj@LAPTOP-FT6VVM4D: $ kill -l
 1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
 6) SIGABRT     7) SIGBUS      8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5 60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX

shreyaj@LAPTOP-FT6VVM4D: $ kill -15 815
-bash: kill: (815) - No such process
shreyaj@LAPTOP-FT6VVM4D: $ kill -15 1178
shreyaj@LAPTOP-FT6VVM4D: $

```

TASK4: To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

```
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ nano signal.c
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ nano resume_jobs.c
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ gcc resume_jobs.c -o resume_jobs
./resume_jobs
Session 9 - Resume Stopped Jobs
Commands: run <command>, jobs, resume <job_id>, exit
myshell> PDF
myshell> run sleep 20
Job [1] started and stopped. PID: 1425
myshell> jobs

Job Table:
-----
Job ID  PID    Status    Command
-----
[1]     1425    Completed  sleep
-----

myshell> resume 1
Job [1] is not in Stopped state (Current: Completed).
myshell> exit
Exiting...
shreyaj@LAPTOP-FT6VVM4D:~/Skill9$ |
```

```
GNU nano 8.7.1 resume_jobs.c
input[strcmp(input, "\n")] = 0; // strip newline

if (strcmp(input, "exit") == 0) {
    printf("Exiting...\n");
    break;
} else if (strcmp(input, "jobs") == 0) {
    print_jobs();
} else if (strcmp(input, "run ", 4) == 0) {
    sscanf(input + 4, "%s %s", cmd, arg);

    pid_t pid = fork();
    if (pid == 0) {
        execlp(cmd, cmd, arg, NULL);
        exit(1);
    } else {
        int status;
        waitpid(pid, &status, 0);

        job_count++;
        jobs_table[job_count - 1].job_id = job_count;
        jobs_table[job_count - 1].pid = pid;
        strcpy(jobs_table[job_count - 1].command, cmd);
        strcpy(jobs_table[job_count - 1].status, "Completed");

        printf("Job [%d] started and stopped. PID: %d\n", job_count, pid);
    }
} else if (strcmp(input, "resume ", 7) == 0) {
    int jid;
    sscanf(input + 7, "%d", &jid);
    if (jid > 0 && jid <= job_count) {
        if (strcmp(jobs_table[jid - 1].status, "Stopped") == 0) {
            kill(jobs_table[jid - 1].pid, SIGCONT);
            strcpy(jobs_table[jid - 1].status, "Running");
            printf("Resumed job [%d]\n", jid);
        } else {
            printf("Job [%d] is not in Stopped state (Current: %s).\n", jid, jobs_table[jid - 1].status);
        }
    }
}

return 0;
}
```